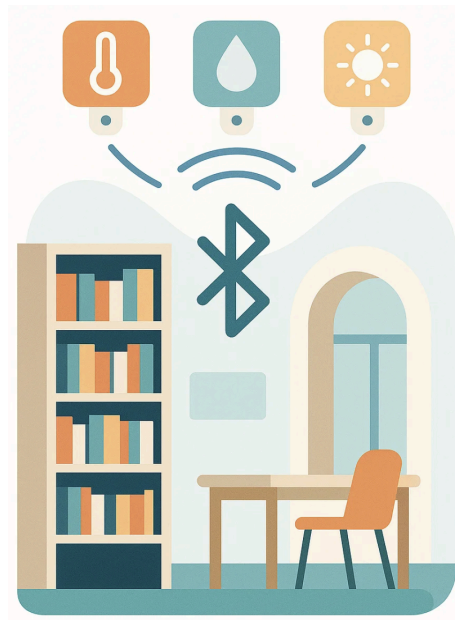




Master's Degree in Electronic Engineering

Integrated Systems Design for IoT (6 CFU)

Microclimate Detection in a Library



Teachers

Prof. Sergio Saponara
Prof. Daniele Rossi
Eng. Ettore Noccetti

Students

Alessio Manuelli
Mattia Bernini
Samuele Bartorelli

Index

1. Introduction.....	2
1.1 Choice of thresholds.....	2
2. Programming.....	3
2.1 Components on the board.....	3
2.1.1 HTS221 Temperature and Humidity Sensor.....	4
2.1.2 BH1750FVI Brightness Sensor.....	8
2.2 Server.....	9
2.3 ESP32 Client.....	12
2.4 Smartphone Client.....	14
2.4.1 Application Interface.....	15
3. Tests and results.....	17
3.1 RSSI as a function of distance and Ptx.....	17
3.2 Power dissipation.....	18
3.2.1 Impact of data rate on power consumption.....	19
4. PIFA antenna.....	21
4.1 Simulation with HFSS.....	21
5. Other solutions on the market.....	26
6. Conclusions.....	27
7. Bibliography.....	28

1. Introduction

The optimal preservation of book collections is a priority for libraries, archives, and museums, especially when it comes to ancient works or items of historical value. These environments require constant monitoring of environmental parameters such as temperature, humidity, and light levels to ensure ideal preservation conditions.

Several commercial solutions for environmental control are available on the market, such as the Testo Saveris 2 system, which utilizes Wi-Fi data loggers and cloud-based architectures. However, many of these solutions are characterized by high costs and closed infrastructures. A more detailed overview is provided in Chapter 5.

In this context, our project proposes a low-power IoT system based on ESP32 microcontrollers and Bluetooth Low Energy (BLE) technology, designed for environments where scalability and direct control are priorities. The goal of the project is to develop a system for monitoring microclimates within a library. Specifically, the system will detect environmental parameters such as temperature, humidity, and light levels using an ESP32 board configured as a server. The collected data will be read by a client, which will compare it with predefined thresholds to ensure they remain within optimal limits—essential for the proper preservation of books.

1.1 Choice of thresholds

Old books (up to the 19th century) and modern books (from the 20th century onwards) are made of different materials, which react differently to environmental factors such as humidity, temperature and light. Consequently, each category requires specific conservation conditions to preserve its integrity over time.

Old books are generally produced with handmade paper, often made of linen or cotton fibers, which are more resistant. The inks used are iron-based, bound with natural materials, while the bindings are made of leather or parchment. On the contrary, modern books are produced with industrial paper, often acidic and derived from wood, use chemical inks and have covers in plasticized or cardboard materials.

Below is a summary table of the ideal environmental conditions for the correct conservation of ancient and modern books, distinguishing the parameters of temperature, humidity and lighting.

Factor	Ancient Books	Modern Books
Temperature	16 - 18 °C	18 - 21 °C
Humidity	45 - 55 %	40 - 60 %
Brightness	< 50 lux	up to 150 lux

Fig. 1: Correct storage conditions

2. Programming

Three codes have been developed: one for the server and two for two different client solutions. The server is based on an ESP32 board that, once the BLE (Bluetooth Low Energy) connection is established, reads data from three sensors on request.

As for the client, two solutions have been proposed:

1. The first involves the use of another ESP32 board, which connects to the server and shows the detected data on the display every 10 seconds.
2. The second one uses a smartphone application, developed by us, which allows you to read data from the server by pressing a button, after establishing a connection.

In both solutions, a warning system has been implemented to notify the user of any exceedances of safety thresholds, via messages displayed on the display.

2.1 Components on the board

Below is an image of the board with the main components used for our project highlighted:

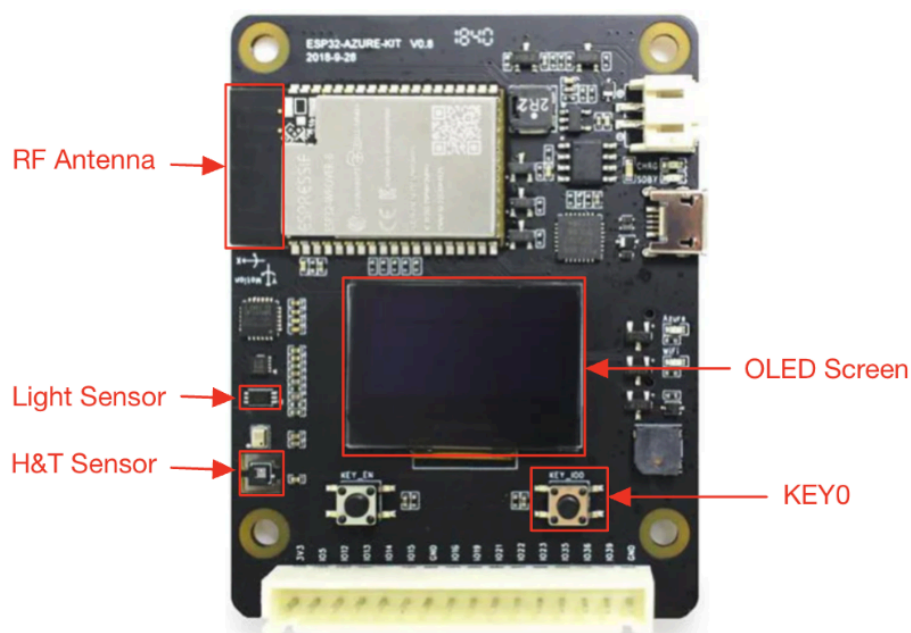


Fig. 2: ESP32 Board

The role of the various components is:

- **RF Antenna:** component responsible for wireless communication.
- **Light Sensor (BH1750FVI):** brightness sensor, which measures the intensity of ambient light in lux.

- **H&T Sensor (HTS221):** combined temperature and humidity sensor.
- **OLED Screen:** OLED display, used to display the values read by the sensors.
- **KEY0:** physical button on the server side; pressing it starts the connection procedure.

2.1.1 HTS221 Temperature and Humidity Sensor

The HTS221 sensor uses I2C interface to communicate and can measure both temperature and relative humidity. Its typical accuracies are ± 0.5 °C for temperature and $\pm 3.5\%$ for humidity.

Below is the I2C address to communicate with the sensor.

```
#define HTS221_SENSOR_ADDR 0x5F
```

Fig. 3: I2C address of the temperature and humidity sensor

To manage the communication with the sensor, two functions were implemented: one to write and one to read the registers. When reading, it was necessary to set bit 7 of the register address to 1 (OR operation with 0x80) to enable the auto-increment mode, useful when reading multiple consecutive bytes.

Below we report the configuration of the various sensor configuration registers:

HTS221_AV_CONF Register (0x10) to which we have assigned the value 0x1B, allows you to average 32 samples for humidity and 16 for temperature. This ensures a more stable reading.

7	6	5	4	3	2	1	0
Reserved	AVGT2	AVGT1	AVGT0	AVGH2	AVGH1	AVGH0	

To configure humidity/temperature average.

Fig. 4: HTS221_AV_CONF Register

HTS221_CTRL_REG1 Register (0x20) to which we assigned the value 0x85, allows the sensor to acquire data 1 time per second, and avoid inconsistent readings between LSB and MSB.

7	6	5	4	3	2	1	0
PD	Reserved				BDU	ODR1	ODR0

[7]	PD: power-down control (0: power-down mode; 1: active mode)
[6:3]	Reserved
[2]	BDU: block data update (0: continuous update; 1: output registers not updated until MSB and LSB reading)
[1:0]	ODR1, ODR0: output data rate selection (see table 17)

Fig. 5: HTS221_CTRL_REG1 Register

The configuration of the various bits is as follows:

- **Bit 7 (PD):** 1 → Active sensor
- **Bit 6–3:** 0000 (not used)
- **Bit 2 (BDU):** 1 → Block Data Update active. Blocks data update until both bytes (MSB and LSB) are read.
- **Bits 1–0 (ODR):** 01 → Output Data Rate of 1 Hz.

Register HTS221_CTRL_REG2 (0x21) to which we have assigned the value 0x80, is used to perform a soft reset of the sensor upon initialization. This ensures that the default settings and internal calibration parameters are reloaded.

7	6	5	4	3	2	1	0
BOOT	Reserved					Heater	ONE_SHOT

[7]	BOOT: Reboot memory content (0: normal mode; 1: reboot memory content)
[6:2]	Reserved
[1]	Heater (0: heater disable; 1: heater enable)
[0]	One-shot enable (0: waiting for start of conversion; 1: start for a new dataset)

Fig. 6: HTS221_CTRL_REG2 Register

The configuration of the various bits is as follows:

- **Bit 7 (BOOT):** 1 → Sensor restart
- **Bit 6–0:** 0000000 → All to 0 (no additional functions active)

Register HTS221_CTRL_REG3 (0x22) to which we have assigned the value 0x00, disables all interrupts. The sensor then operates in polling mode, that is, the data is read manually via the microcontroller.

Temperature measurement

To measure the temperature with the HTS221 sensor, the `hts221_get_temperature()` function has been implemented. This function reads the raw data from the sensor registers, retrieves the calibration parameters and calculates the real temperature through linear interpolation. The steps performed by the function and the registers used are described below.

1. Reading the raw temperature data (T_OUT)

The first step is to read the 2 bytes of the current raw temperature value. The registers involved are:

- **HTS221_TEMP_OUT_L (0x2A)** containing the least significant byte (LSB)

- **HTS221_TEMP_OUT_H (0x2B)** containing the most significant byte (MSB)

These two bytes are combined into a single 16-bit signed integer T_{out} value, representing the raw measurement of the sensor.

2. Reading the raw calibration values (T0_OUT and T1_OUT)

The sensor provides two known reference points (T0 and T1) for calibration. We then read:

- **HTS221_T0_OUT_L (0x3C)** and **HTS221_T0_OUT_H (0x3D)** for the reconstruction of the $T0_{out}$ value.
- **HTS221_T1_OUT_L (0x3E)** and **HTS221_T1_OUT_H (0x3F)** for the reconstruction of the $T1_{out}$ value.

These are also raw 16-bit signed values, representing the output value the sensor produces at two known temperatures.

3. Reading the calibration values in °C (T0 and T1)

The sensor does not directly provide the temperatures T0 and T1 in degrees Celsius, but scaled values that must be reconstructed:

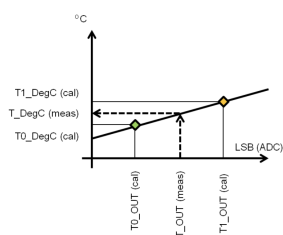
- **HTS221_T0_degC_x8 (0x32)**: 8 bits LSB of T0, multiplied by 8
- **HTS221_T1_degC_x8 (0x33)**: 8 bit LSB of T1, multiplied by 8
- **HTS221_T1_T0_MSB (0x35)**: contains the 2 MSB bits of T0 (bit 1:0) and T1 (bit 3:2)

Combining the LSB and MSB, we reconstruct the full 10-bit values and divide by 8.0 to get the temperatures in degrees Celsius:

- $T0_{degC} = T0_x8 / 8.0$
- $T1_{degC} = T1_x8 / 8.0$

4. Calculating the real temperature

Finally, a linear interpolation is applied between the two calibration points to calculate the actual temperature as reported in the datasheet:



$$\text{temperature} = (T1_{degC} - T0_{degC}) \cdot \frac{T_{out} - T0_{out}}{T1_{out} - T0_{out}} + T0_{degC}$$

Fig. 7: Linear interpolation for the calculation of the real temperature

Humidity measurement

For the humidity sensor the steps are very similar. For the measurement the function `hts221_get_humidity()` was created. This reads the raw data from the sensor, uses the calibration parameters stored in the internal registers and calculates the real humidity value through a linear interpolation.

1. Reading the raw humidity data (H_OUT)

The first step is to read the current raw moisture value. The registers involved are:

- **HTS221_HUM_OUT_L (0x28)** containing the least significant byte (LSB)
- **HTS221_HUM_OUT_H (0x29)** containing the most significant byte (MSB)

The two bytes are combined to obtain the `H_out` value, a 16-bit signed integer representing the raw measurement of humidity.

2. Reading the raw calibration values (H0_OUT and H1_OUT)

The HTS221 sensor includes two internal calibration points for humidity, H0 and H1. Their raw values are found in the following registers:

- **HTS221_H0_T0_OUT_L (0x36)** and **HTS221_H0_T0_OUT_H (0x37)** for the reconstruction of the `H0_out` value
- **HTS221_H1_T0_OUT_L (0x3A)** and **HTS221_H1_T0_OUT_H (0x3B)** for the reconstruction of the `H1_out` value

These are also signed 16-bit values, and represent the sensor output at the two known calibration points.

3. Reading the calibration values in %RH (H0 and H1)

Unlike temperature, for humidity the calibration values in %RH are already stored in a simplified form: multiplied by 2 and in 8 bits.

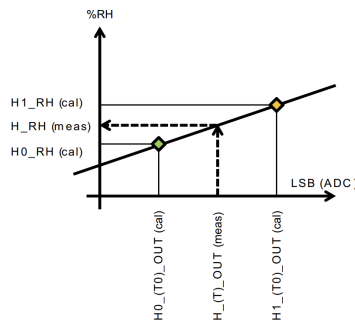
The registers used are:

- **HTS221_H0_RH_x2 (0x30)** which contains the value of H0 in the format $\%RH * 2$
- **HTS221_H1_RH_x2 (0x31)** which contains the value of H1 in the format $\%RH * 2$

The values are converted to the corresponding `H0_RH` and `H1_RH` in percentage by dividing by 2.0.

4. Calculating the real humidity

Once all the parameters are obtained, the calibration formula is applied:



$$\text{humidity} = (H1_{RH} - H0_{RH}) \cdot \frac{H_{out} - H0_{out}}{H1_{out} - H0_{out}} + H0_{RH}$$

Fig. 8: Linear interpolation for the calculation of real humidity

This linear interpolation converts the raw H_out value into a real relative humidity percentage value.

To comply with the sensor specifications (as indicated in the datasheet), the obtained value is limited between 0.0% and 100.0% to compensate for any deviations due to measurement or calibration errors.

2.1.2 BH1750FVI Brightness Sensor

The BH1750FVI sensor uses the I2C interface to communicate and can measure ambient brightness with high accuracy, up to 0.5 lux in High Resolution mode.

Below is the I2C address used to communicate with the sensor.

```
#define BH1750FVI_SENSOR_ADDR 0x23
```

Fig. 9: I2C address of the light sensor

To manage the communication with the BH1750FVI, specific commands are used. These commands are sent via I2C writing and it is not necessary to set special bits for auto-increment.

Below are the sensor configuration commands used:

BH1750FVI_POWER_ON (0x01): Command to turn on the sensor. It is used to exit Power Down mode and prepare the sensor to receive measurement commands.

Power On	0000_0001	Waiting for measurement command.
----------	-----------	----------------------------------

Fig. 10: Sensor power on command

BH1750FVI_CONTINUOUS_MESAURE_HR (0x10): Command to start continuous measurement in High Resolution mode.

Continuously H-Resolution Mode	0001_0000	Start measurement at 1lx resolution. Measurement Time is typically 120ms.
--------------------------------	-----------	--

Fig. 11: Measurement start command

To measure the brightness with the BH1750FVI sensor, the function `bh1750fvi_get_luminosity()` was created. This function performs the following steps:

1. **Reading raw data:** The function reads 2 bytes from the sensor via the `bh1750fvi_read()` function, which performs I2C communication with the BH1750FVI to get the raw brightness value.
2. **Reconstruction of the 16-bit value:** The two read bytes are combined into a single 16-bit value (`Lx_aux`) with a left shift of the most significant byte and an OR operation with the least significant byte.
3. **Conversion to Lux:** The raw value is converted to Lux by dividing by 1.2, as indicated by the sensor datasheet. This conversion normalizes the raw data to a real value.
4. **Value saturation:** a saturation is applied to avoid values higher than 600 Lux. If the calculated value exceeds 600 Lux, it is forced to 600. This avoids having values that cannot be represented on 16 bits. In our case the value 600 is more than fine since the maximum threshold at which books can be stored is 150.

2.2 Server

When you start the card, you will see the following initial screen:



Fig. 12: Home Screen (Server)

To start the connection search with the client, you need to press the KEY0 button located on the board. Until the connection is established, the following message will be shown on the display:



Fig. 13: Waiting for connection screen (Server)

Once the connection is established, the server, upon request from the client, performs measurements of the requested quantities, displaying the values read on the display as shown below:

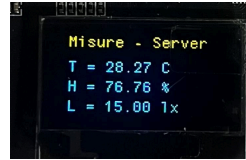


Fig. 14: Presentation screen after connection established (Server)

To transmit data, the board uses the BLE protocol, a low-power wireless technology designed for applications that require frequent but short-range communications. In particular, the system relies on the GAP and GATT roles.

Initializing the Bluetooth stack and configuring the driver callback functions are located in main:

```
void app_main(void)
{
    ESP_ERROR_CHECK(nvs_flash_init()); // Inizializza la partizione NVS (Non-Volatile Storage) nella memoria flash.

    ESP_ERROR_CHECK(i2c_master_init()); // Inizializza il controller I2C come master per comunicare con i sensori

    wait_for_button_press(); // Attende la pressione di un pulsante prima di procedere con l'inizializzazione del BLE

    // Configura il sensore HTS221 (temperatura e umidità) scrivendo i valori appropriati nei suoi registri
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_AV_CONF, HTS221_AV_CONF_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG1, HTS221_CTRL_REG1_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG2, HTS221_CTRL_REG2_VAL));
    ESP_ERROR_CHECK(HTS221_register_write_byte(HTS221_CTRL_REG3, HTS221_CTRL_REG3_VAL));

    // Configura il sensore BH1750FVI (luminosità) per l'accensione e la misurazione continua in alta risoluzione
    ESP_ERROR_CHECK(BH1750FVI_write_byte(BH1750FVI_POWER_ON));
    ESP_ERROR_CHECK(BH1750FVI_write_byte(BH1750FVI_CONTINUOUS_MEASURE_HR));

    esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_bt_controller_init(&bt_cfg)); // Inizializzazione del controller Bluetooth (HCI, Link Layer e Physical Layer)
    ESP_ERROR_CHECK(esp_bt_controller_enable(ESP_BT_MODE_BLE)); // Abilita il controller Bluetooth in modalità BLE (Bluetooth Low Energy)
    ESP_ERROR_CHECK(esp_btdev_init()); // Inizializzazione dello stack software Bluetooth (crea il thread gestore del Bluetooth).
    ESP_ERROR_CHECK(esp_btdev_enable()); // Abilita lo stack Bluetooth.

    // Registra le funzioni di callback per gli eventi GAP e GATT. Queste funzioni verranno chiamate dal driver BLE quando si verificano eventi rilevanti
    ESP_ERROR_CHECK(esp_ble_gap_register_callback(esp_gap_cb));
    ESP_ERROR_CHECK(esp_ble_gatts_register_callback(esp_gatts_cb));

    ESP_ERROR_CHECK(esp_ble_gap_set_device_name("IOT_SENSOR")); // Imposta il nome del dispositivo BLE che sarà visibile durante l'advertising
    ESP_ERROR_CHECK(esp_ble_tx_power_set(ESP_BLE_PWR_TYPE_ADV, ESP_PWR_LVL_N12)); // Imposta il livello di potenza di trasmissione BLE per l'advertising
    ESP_ERROR_CHECK(esp_ble_gatts_app_register(PROFILE_APP_ID)); // Registra l'applicazione GATT con l'ID del profilo.
    //Questo innescherà l'evento ESP_GATTS_REG_EVT nella callback GATT
}
```

Fig. 15: Bluetooth stack initialization code (Server)

The Generic Access Profile (GAP) manages how the server presents itself and communicates its presence to other devices. The device advertises itself with parameters defined in `ble_adv_params` and data in `adv_data`.

```
// Parametri di advertising BLE. Controllano come il dispositivo si pubblicizza agli altri dispositivi BLE
static esp_ble_adv_params_t ble_adv_params = {
    .adv_int_min = 0x20, // Intervallo minimo di advertising (multiplo di 0.625ms, qui 32 * 0.625ms = 20ms)
    .adv_int_max = 0x40, // Intervallo massimo di advertising (multiplo di 0.625ms, qui 64 * 0.625ms = 40ms)
    .adv_type = ADV_TYPE_IND, // Tipo di advertising: connettabile e scansionabile
    .own_addr_type = BLE_ADDR_TYPE_PUBLIC, // Tipo di indirizzo BLE utilizzato (pubblico)
    .channel_map = ADV_CHNL_ALL, // Mappa dei canali di advertising (tutti i 3 canali primari)
    .adv_filter_policy = ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY, // Policy di filtro per le scansioni e le connessioni (consente a tutti)
};

// Dati di advertising BLE. Definiscono quali informazioni saranno incluse nei pacchetti di advertising
static esp_ble_adv_data_t adv_data = {
    .include_name = true, // Includi il nome del dispositivo nei dati di advertising
    .include_txpower = true, // Includi il livello di potenza di trasmissione nei dati di advertising
    .flag = ESP_BLE_ADV_FLAG_LIMIT_DISC | ESP_BLE_ADV_FLAG_BREDR_NOT_SPT, // Flag specifici: limitatamente scansionabile, BR/EDR non supportato (solo BLE)
};
```

Fig. 16: Advertising parameters and data

The minimum and maximum advertising intervals have been set to 20 ms and 40 ms respectively. The actual advertising interval will be randomly chosen between the minimum and maximum to prevent collisions with other devices.

esp_gap_cb is the callback that receives GAP events like:

- **ESP_GAP_BLE_ADV_DATA_SET_COMPLETE_EVT**: when advertising data is set.
- **ESP_GAP_BLE_ADV_START_COMPLETE_EVT**: when advertising starts.
- **ESP_GAP_BLE_ADV_STOP_COMPLETE_EVT**: when advertising stops.

Starts and stops advertising to allow or suspend connection from a client.

GATT, on the other hand, defines how data (attributes) are organized, read and written on the device. In our code we created a GATT service with UUID 0x00FF. We then defined three characteristics:

- **Temperature**(CHAR_UUID_TEMP)
- **Humidity**(CHAR_UUID_HUMI)
- **Brightness**(CHAR_UUID_LUX)

For each feature, we set permissions (read-only) and initial values.

The **esp_gatts_A_cb** callback handles GATT events such as:

- Profile registration,
- Creating and starting the service,
- Adding features,
- Characteristics reading management (ESP_GATTS_READ_EVT),
- Client connection and disconnection.

When a client reads a feature, the code reads the value from the sensor, converts it to a suitable format, and responds to the client.

The data read by the sensor is appropriately converted before being sent to the client as shown below:

```

// Converte il valore float del sensore in un int16_t moltiplicandolo per 100 e lo divide in due byte (LSB e MSB) per l'invio via BLE.
uint16_t val = (uint16_t)(sensor_value * 100);
data[0] = val & 0xFF; // Byte meno significativo
data[1] = (val >> 8) & 0xFF; // Byte più significativo

// Prepara la risposta GATT da inviare al client
esp_gatt_rsp_t rsp;
memset(&rsp, 0, sizeof(rsp)); // Inizializza la struttura a zero
rsp.attr_value.handle = handle; // Handle della caratteristica che è stata letta
rsp.attr_value.len = 2; // Lunghezza della risposta (2 byte)
rsp.attr_value.value[0] = data[0]; // LSB del valore
rsp.attr_value.value[1] = data[1]; // MSB del valore

// Invia la risposta GATT al client (non avendo messo la risposta automatica questa va inviata manualmente)
ESP_ERROR_CHECK(esp_ble_gatts_send_response(
    gatts_if, // Interfaccia GATT
    param->read.conn_id, // ID della connessione
    param->read.trans_id, // ID della transazione
    ESP_GATT_OK, // Stato della risposta (OK)
    &rsp)); // Puntatore alla struttura della risposta

```

Fig. 17: Conversion code of the read data

The value read from a sensor is first multiplied by 100 (to avoid using a comma). The result of the multiplication is then cast to `uint16_t`. This means that the value will be a 16-bit unsigned integer, with a range of 0 to 65535.

After that, the two bytes `data[0]` and `data[1]` are prepared and sent.

A variable `rsp` of type `esp_gatt_rsp_t` is then declared. This is a structure defined by the ESP-IDF that encapsulates all the details needed to construct a response to a GATT request from the client.

The two bytes (`data[0]` and `data[1]`) representing the sensor value are copied into the `value[0]` and `value[1]` fields of the `rsp.attr_value.value` array. This array is where the actual feature data is stored to be sent.

At this point we have the GATT response sent to the client via the `esp_ble_gatts_send_response()` function.

2.3 ESP32 Client

When the program starts, the scanning phase begins. Until the connection is established, the following message will be shown on the display:

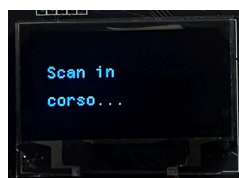


Fig. 18: Waiting for connection screen (Client)

Once the connection is established, the client requests the temperature, humidity, and pressure measurements. These are received every 10 seconds, and are displayed as follows:

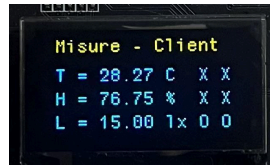


Fig. 19: Presentation screen after connection established (Client)

The two columns represent respectively: the left column the case of ancient books, and the right column that of modern books. If the detected value falls within the expected threshold, it is indicated with an 'O', otherwise, with an 'X'.

As with the server, initializing the Bluetooth stack and configuring the driver callback functions are found in main:

```
void app_main(void)
{
    ESP_ERROR_CHECK(nvs_flash_init());
    ESP_ERROR_CHECK(i2c_master_init());
    ESP_ERROR_CHECK(esp_bt_controller_mem_release(ESP_BT_MODE_CLASSIC_BT));

    //Inizializzazione controller Bluetooth (HCI, LL and PHY)
    esp_bt_controller_config_t bt_cfg = BT_CONTROLLER_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_bt_controller_init(&bt_cfg));
    ESP_ERROR_CHECK(esp_bt_controller_enable(ESP_BT_MODE_BLE));

    //Inizializzazione Stack Software (Creazione thread gestore del bluetooth)
    ESP_ERROR_CHECK(esp_bluedroid_init());
    ESP_ERROR_CHECK(esp_bluedroid_enable());

    printf("Stack Initialized\n\n");

    //Configurazione della funzione di callback del driver
    ESP_ERROR_CHECK(esp_ble_gap_register_callback(esp_gap_cb));
    ESP_ERROR_CHECK(esp_ble_gattc_register_callback(esp_gattc_cb));

    ESP_ERROR_CHECK(esp_ble_gattc_app_register(PROFILE_A_APP_ID));
}
```

Fig. 20: Bluetooth stack initialization code (Client)

Once the connection with the server is established, at a certain point, the event for reading the characteristics of temperature, humidity and pressure is generated. Regarding the temperature:

```
case ESP_GATTC_SEARCH_CMPL_EVT:
{
    printf("ESP_GATTC_SEARCH_CMPL_EVT\n");
    if(get_server == true) { // Se il servizio è stato trovato con successo.
        uint16_t count = 0;
        esp_gattc_char_elem_t *char_elem_result = NULL;

        ESP_ERROR_CHECK(esp_ble_gattc_get_attr_count(gattc_if, param->search_cmpl.conn_id, ESP_GATT_DB_CHARACTERISTIC,
            gl_profile_tab[PROFILE_A_APP_ID].service_start_handle,
            gl_profile_tab[PROFILE_A_APP_ID].service_end_handle, IGNORE_HANDLE, &count));

        if (count > 0) {
            char_elem_result = (esp_gattc_char_elem_t *) malloc(sizeof(esp_gattc_char_elem_t) * count);
            if (char_elem_result) {
                // Cerca la caratteristica specifica della temperatura tramite UUID.
                ESP_ERROR_CHECK(esp_ble_gattc_get_char_by_uuid(gattc_if, param->search_cmpl.conn_id, gl_profile_tab[PROFILE_A_APP_ID].service_start_handle,
                    gl_profile_tab[PROFILE_A_APP_ID].service_end_handle, remote_char_uuid_temp, char_elem_result,
                    &count));

                if (count > 0) {
                    // Salva l'handle della caratteristica temperatura.
                    gl_profile_tab[PROFILE_A_APP_ID].char_handle_temp = char_elem_result[0].char_handle;
                    printf("Temperatura Handle: %d\n", gl_profile_tab[PROFILE_A_APP_ID].char_handle_temp);
                }
            }
        }
    }
}
```

Fig. 21: Sent data reading code (temperature)

An appropriate amount of dynamic memory is allocated. Then, starting from the UUID, the characteristic of interest is found, and the information about it is saved in the previously allocated memory space. This is done for all three characteristics. At this point, the handles can be saved and the three characteristics can be read.

To reconstruct and convert the value transmitted by the server, proceed as follows:

```
// Il server invia un uint16_t (due byte) rappresentante il valore del sensore moltiplicato per 100.  
// Ricostruisce il valore uint16_t dal LSB e MSB.  
uint16_t raw_value = (param->read.value[1] << 8) | param->read.value[0]; // Ricostruisce il valore grezzo (LSB, poi MSB).  
// Converte il valore grezzo in un float dividendo per 100.  
float sensor_value = (float)raw_value / 100.0f; // Converte in float e divide per 100.
```

Fig. 22: Actual data reconstruction code

The first step is to reconstruct the 16-bit value by combining the two bytes received. Once the raw value is obtained, it is converted to floating point (float) and divided by 100. This division serves to bring the number back to its real value, compensating for the multiplication performed by the server when sending.

Finally, to represent the values on the display, a function called `show_value` is defined. This is responsible for providing the screen shown in the previous figure.

2.4 Smartphone Client

For the smartphone client we implemented an iPhone application using the Xcode program. In particular, the code defines a ViewController class that manages the interaction with the server.

The program uses the UIKit framework for the graphical interface and CoreBluetooth for BLE communication. The ViewController inherits from UIViewController and implements the CBCentralManagerDelegate and CBPeripheralDelegate delegate protocols to handle Bluetooth-related events, such as scanning, connecting, and reading data from the peripheral device.

The main flow is as follows:

1. Upon startup, it initializes the Bluetooth manager (CBCentralManager) and starts scanning for BLE devices.
2. When a device with the name "IOT_SENSOR" is discovered, scanning stops and a connection is attempted.
3. Once connected, specific services and characteristics identified by custom UUID are searched for (service 00FF, characteristics FF01, FF02, FF03 for temperature, humidity and brightness).
4. The device periodically reads the RSSI value to update the signal quality.

- When value updates are received from sensors, they are converted, evaluated against configurable thresholds via segmented control, and displayed in the UI with colored text indicating whether the value is “OK” or out of range.
- You can manually read the data by pressing a button, or connect/disconnect by another button.
- The parameter evaluation compares the values with predefined thresholds, highlighting any anomalies with a red color and a warning icon.

2.4.1 Application Interface

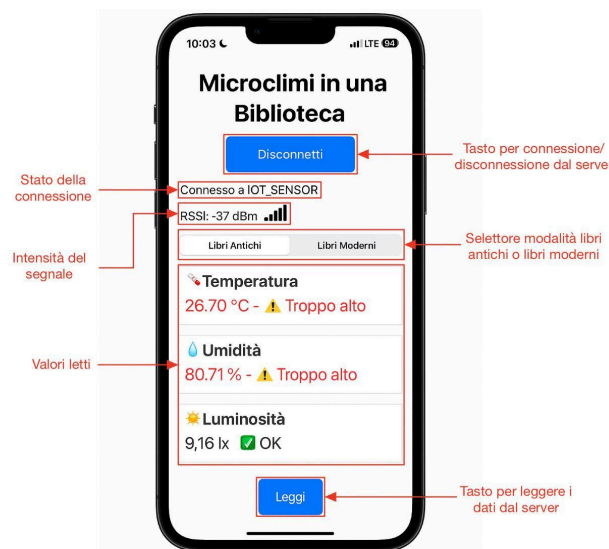


Fig. 23: Application Interface

At the top of the interface we find the button to connect or disconnect the device from the server, identifiable by the blue “Disconnect” button. Once the connection is established, the app shows the status with the words “Connected to IOT_SENSOR”, confirming to the user that communication with the IoT sensor is active.

Immediately below the connection status is the RSSI (Received Signal Strength Indicator) value, expressed in dBm. This indicator provides a quantitative measure of the strength of the radio signal received by the device. Next to it, there is an icon that graphically represents the quality of the signal, divided into six levels, facilitating an immediate evaluation also at a visual level.

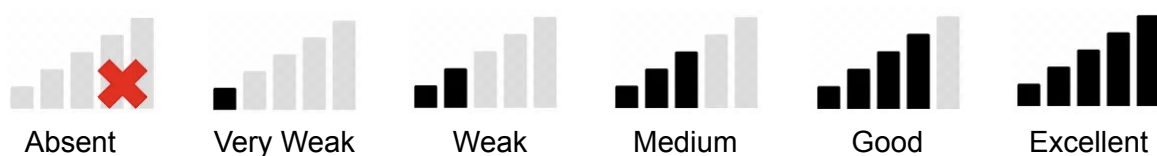


Fig. 24: Signal quality indices

The thresholds were chosen as follows:

- **Absent:** connection not established or RSSI < - 90 dBm
- **Very Weak:** - 80dBm < RSSI < - 90dBm
- **Weak:** - 70dBm < RSSI < - 80dBm
- **Medium:** - 60dBm < RSSI < - 70dBm
- **Good:** - 50dBm < RSSI < - 60dBm
- **Excellent:** - 40dBm < RSSI < - 50dBm

The interface includes a two-option selector to choose whether to monitor the ideal conditions for Old Books or Modern Books.

The central area of the interface shows the values read by the sensor, divided into three main categories, one for each value read.

Finally, at the bottom there is the "Read" button, which allows the user to manually read the data.

3. Tests and results

Some tests have been done to evaluate RSSI as a function of distance and transmitted power. The transmitted power of the server can be modified directly from the code using the following line:

```
ESP_ERROR_CHECK(esp_ble_tx_power_set(ESP_BLE_PWR_TYPE_ADV, ESP_PWR_LVL_N12));
```

Fig. 25: Selecting the transmission power value

For normal operation it was set to -12 dBm, later modified for further testing.

Additionally, tests were conducted on server power consumption during advertising and communication activities.

3.1 RSSI as a function of distance and Ptx

This test is used to evaluate how the RSSI value varies with the distance and transmission power of the server. In particular, two types of tests were performed:

1. **In Line of Sight (LoS) conditions:** 4 different power levels and 4 different distances were used.
2. **In Non Line of Sight (NLoS) conditions:** with the transmission power set at -12 dBm, an obstacle (a wall) was inserted between the server and the client and the measurements were repeated at the same distances.

Below are the results obtained.

LoS

txPower [dBm]	RSSI [dBm] @ 1m	RSSI [dBm] @ 3m	RSSI [dBm] @ 6m	RSSI [dBm] @ 9m
-12	-63	-65	-75	-79
-6	-60	-64	-68	-71
0	-59	-62	-65	-69
6	-57	-60	-62	-67

Fig. 26: RSSI in LoS conditions

NLoS

txPower [dBm]	RSSI [dBm] @ 1m	RSSI [dBm] @ 3m	RSSI [dBm] @ 6m	RSSI [dBm] @ 9m
-12	-66	-74	-77	-81

Fig. 27: RSSI under NLoS conditions

As we can see, RSSI decreases with increasing distance, as expected. This behavior is consistent with the free-space signal attenuation model. For the same distance, increasing the transmission power leads to an improvement in RSSI.

By entering the obstacle the RSSI is systematically lower than the corresponding LoS values.

3.2 Power dissipation

The goal of this analysis is to evaluate the energy consumption of the ESP32 module in the two operating phases. An estimate of the battery life is also made based on the average dissipated power.

The tests were performed exclusively on the server side. The client, in fact, is considered to be powered by a stable energy source, such as the electrical network, and therefore not subject to autonomy constraints. The server, on the other hand, could reasonably be powered by a battery.

The tests were performed in the following scenarios:

1. **BLE Advertising:**the card is constantly advertising.
2. **Active BLE connection:**the card is connected to a client and exchanges data.

Below is a table summarizing the results obtained:

State	Power [mW]	Power [mW]
Advertising	161 - 345	253
Connection	161 - 485	323

Fig. 28: Power dissipation (Server)

It can be observed how there is a range of dissipated power during the various moments of the test.

To estimate the battery life of the ESP32 system, we assume that we are using a commercial coin cell battery, such as a CR2032, which is commonly used in low-power devices. This battery provides a nominal voltage of about 3.0 V and has a typical capacity of 220 mAh, which is equivalent to 660 mWh of available energy.

To estimate battery life, only the connection-related contribution is taken into account. In fact, the advertising phase lasts a few seconds, while the rest of the time there is actual communication between client and server.

Once the average power is estimated, the estimated battery life is obtained by:

$$t_{lifetime} = \frac{E_{battery}}{P_{avg}} \quad (1)$$

Where $E_{battery}$ is the energy available from the battery, calculated as:

$$E_{battery} = V \cdot Q = 3 V \cdot 0.220 Ah = 0.66 Wh = 660 mWh \quad (2)$$

So in our case:

$$t_{lifetime} = \frac{660 mWh}{323 mW} \simeq 2h \quad (3)$$

The power consumption evaluation was performed with the display turned on. This choice makes sense only at the design level, both for aesthetic reasons and to facilitate any debugging operations. However, with a view to further energy saving, it is obvious that the display can be turned off during normal operation.

An effective way to increase energy efficiency is to implement a sleep mode for the edge node during periods when it does not need to acquire or transmit data to the central node.

3.2.1 Impact of data rate on power consumption

In our BLE system, during the connection, the server sends data to the client on request. Each request involves the transmission of 6 bytes in total, divided into 3 characteristics of 2 bytes each.

The time required to transmit the 6 bytes depends on the BLE data rate. Considering a data rate of 1 Mbps:

- 1 Mbps = 1 million bits per second
- 1 (Preamble) + 4 (Access Address) + 2 (Header) + 2 (Payload) + 3 (CRC) = 12 bytes
- 12 * 3 = 36 bytes → 288 bits

- Transmission time = $288 \text{ bits} \div 1,000,000 \text{ bits/s} = 288 \text{ } \mu\text{s}$ (microseconds)

In practice, the transmission lasts less than about 0.3 ms.

In the project, we considered the worst-case scenario, namely the transmission of two bytes for each value of every characteristic. To reduce the data rate and therefore the power consumption during transmission the system can be optimized by decreasing the number of bytes sent, for example by reducing the resolution of the values.

4. PIFA antenna



Fig. 29: PIFA antenna

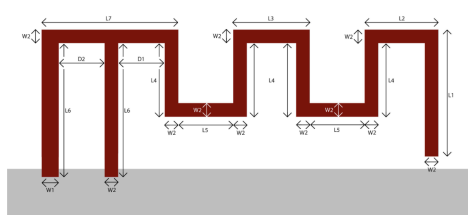
The PIFA is a variation of the inverted "F" antenna. On PCB, it appears as a copper metal trace shaped into a "meander" shape.

It is composed of:

- **Radiant element:** the copper strip on the PCB.
- **Ground plan.**
- **Short circuit:** connects the radiating element to the ground plane.
- **Diet:** where the antenna is connected to the RF circuit of the ESP32 module.

4.1 Simulation with HFSS

The first operation carried out was the design of the ground plane, in copper and the substrate, in FR4, using standard thicknesses, i.e. 70 μm for the ground plane and 1.6 mm for the substrate. Subsequently, we designed the antenna based on the measurements found online reported below.



Reference	mm
L1	4.67 mm
L2	2.7 mm
L3	2.8 mm
L4	2.7 mm
L5	2 mm
L6	4.91 mm
L7	5 mm
W1	0.63 mm
W2	0.5 mm
D1	1.7 mm
D2	1.67 mm

Fig. 30: Antenna characteristic measurements

The length of the final "leg" L1 has been assigned the parameter 'L' in order to perform subsequent sweeps.

The final result is as follows:

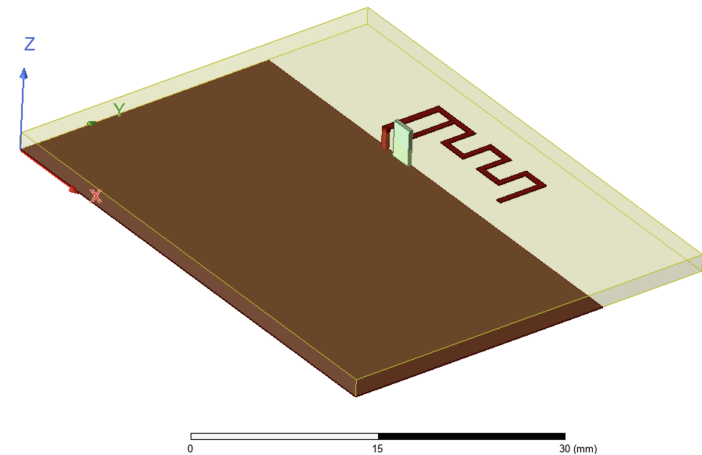


Fig. 31: Antenna design

In particular, one of the antenna terminals was connected to the ground plane, while the other was connected to a wave port, which represents the point where the signal to be transmitted arrives.

We then simulated the antenna's reflection coefficient S_{11} , obtaining a resonant frequency of 2.3 GHz, slightly lower than the expected value of 2.4 GHz.

To correct this discrepancy, a sweep of the length L was performed. The first sweep was performed with L values between 1 mm and 8 mm, with a step of 1 mm. The result obtained is the following:

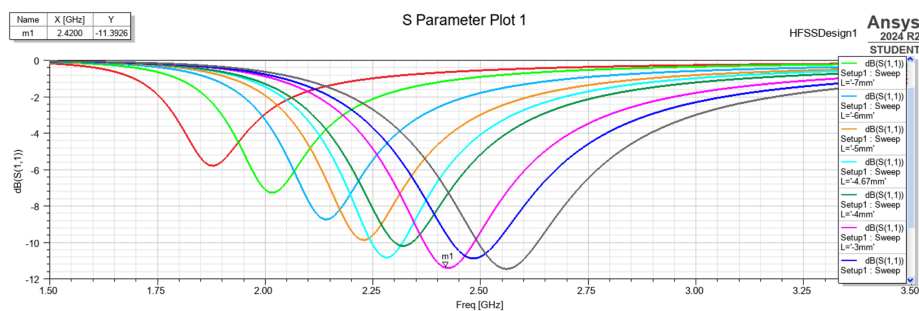


Fig. 32: Frequency trend of the reflection coefficient S_{11} for different lengths (1)

It is observed that the optimal point is around 3 mm. At this point, we performed a finer sweep, varying L from 2.5 mm to 3.5 mm with increments of 0.2 mm, to identify the optimal value more precisely. The results obtained are the following:

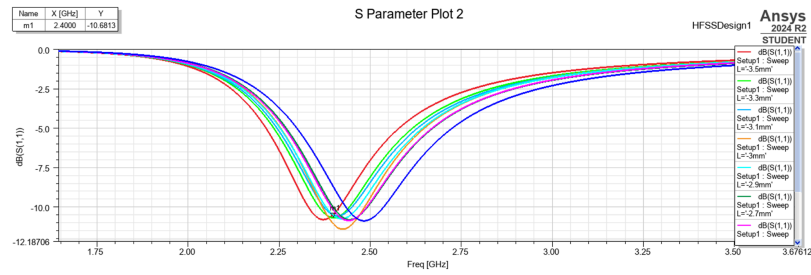


Fig. 33: Frequency trend of the reflection coefficient S_{11} for different lengths (2)

Simulations show that the optimal value of L is 3.3 mm, at which resonance is obtained exactly at 2.4 GHz.

With $L = 3.3$ mm we finally determined the minimum of S_{11} and the bandwidth of the antenna, setting a marker at -10 dB. We thus obtained a minimum of -10.68 dB and a bandwidth between 2.3598 GHz and 2.4425 GHz.

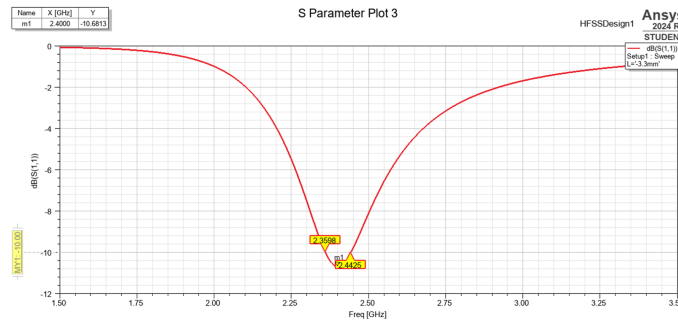


Fig. 34: Determination of the antenna passband

So far we have optimized the length L by focusing only on the parameter S_{11} , without considering the input matching. To obtain better results, we can try to match the antenna impedance to that of the wave port.

We then evaluated the antenna input impedance at the frequency of 2.4 GHz, obtaining the following results:

Real Part: 94.72 Ω

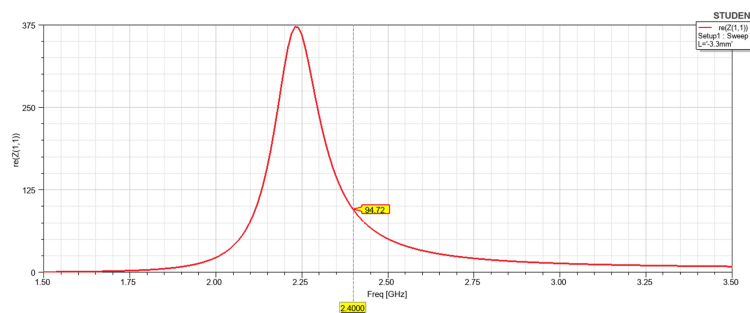


Fig. 35: Frequency trend of the input impedance (real part) of the antenna

Imaginary Part: -60.50Ω

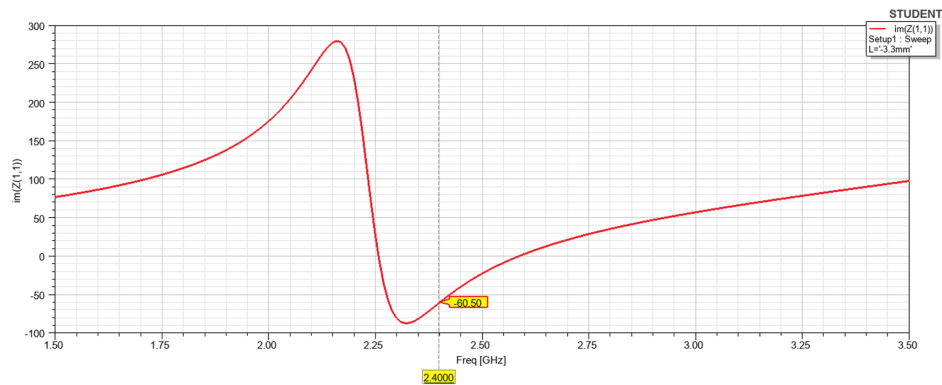


Fig. 36: Frequency trend of the input impedance (imaginary part) of the antenna

From this, it turns out: $Z_{11} = 94.72 - 60.50j$.

By setting the source impedance equal to the complex conjugate of the antenna impedance we obtain the following result:

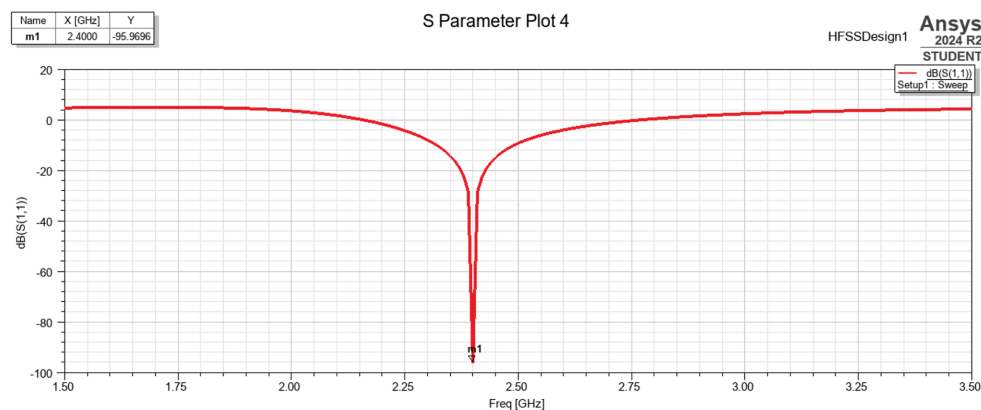


Fig. 37: Frequency trend of the reflection coefficient S_{11} under DC matching conditions

We note a clear improvement in the S_{11} parameter compared to the previous case.

Alternatively, it would have been possible to modify various parameters, such as the length of the leg at which the RF signal is injected, in order to obtain an antenna impedance of 50Ω . Or, one could simply insert a matching network. This may not be necessary if, for example, the antenna designer designs the circuit that uses the antenna.

Below is the 3D and 2D representation of the radiation pattern:

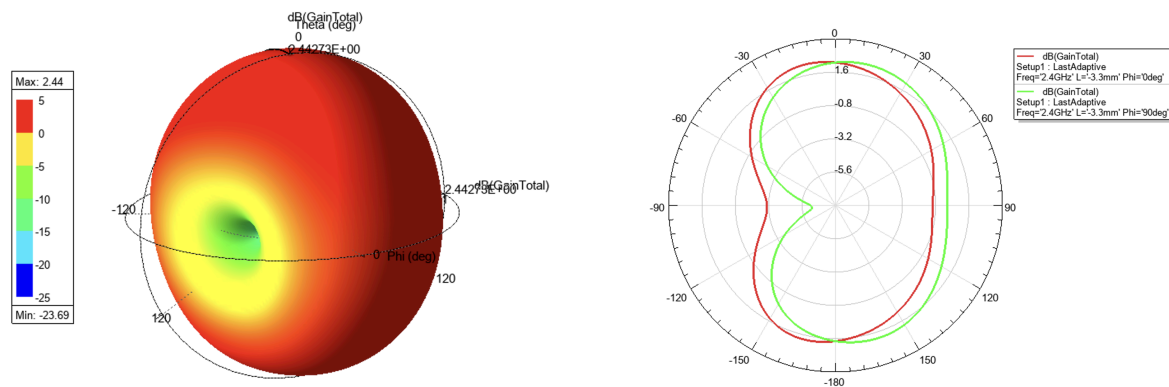


Fig. 38: Radiation diagrams

The three-dimensional representation of the total gain (in dB) is shown on the left, showing a maximum of about 2.44 dB. The distribution is typical of a PIFA, with a directional pattern and good coverage in the desired direction.

On the right are the radiation patterns in polar coordinates in the planes $\phi=0$ (in red) and $\phi=90$ (in green).

5. Other solutions on the market

There are solutions on the market for monitoring environments such as libraries. One example is Testo Saveris 2, which is a Wi-Fi data logger system for monitoring temperature and humidity.

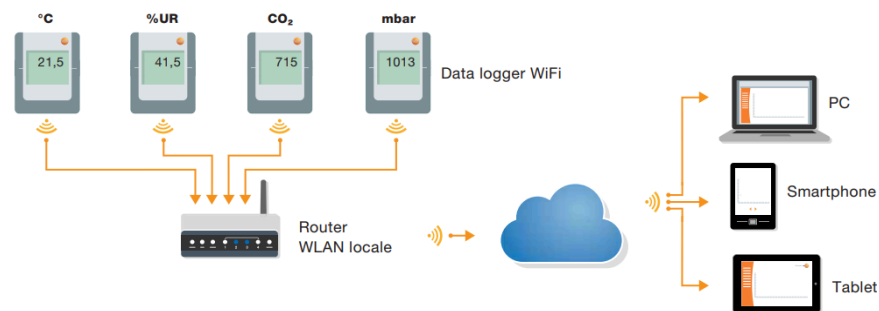


Fig. 39: *Generic environmental monitoring system*

Small devices are used to continuously measure temperature, humidity, and/or other parameters. These devices automatically send the measured data to a dedicated Testo cloud at configurable intervals (e.g. every 5, 10, 15 minutes). This allows the data to be accessed from anywhere, as long as there is internet access.

With this system it is possible to perform temperature and humidity measurements with tolerances of ± 0.5 °C and $\pm 2\%$. A battery life of 12 months is expected, at +25 °C, with a measurement cycle of 15 min.

6. Conclusions

The project allowed us to analyze in detail the operation of BLE, with particular attention to energy consumption. By studying the absorbed power and comparing it with the battery life, we understood how important it is to adopt low power and low energy techniques in applications of this type, where energy saving is a determining factor.

7. Bibliography

- **Expressif (ESP32):** <https://www.espressif.com/en/products/socs/esp32>
- **HTS221:** <https://www.st.com/resource/en/datasheet/hts221.pdf>
- **BH1750FVI:** <https://www.mouser.com/datasheet/2/348/bh1750fvi-e-186247.pdf?srlti d=AfmBOopCUhfmkBHbkjVxZYpOLc8UI8e3GIMVhfBNxWPChW9nPpujlnpb>
- **PIFA:** https://github.com/prasad-dot-ws/ESP32_MIFA_PCB_ANTENNA
- **Saveris 2 Text:** <https://www.testo.com/it-IT/applications/museum-saveris-2>